

Data Analytics Project

Sales & Order Management Analytics System

A practical SQL project covering relational database design, table relationships, CRUD operations, joins, aggregations, subqueries, CTEs, window functions, views, and business analytics.

Project Description

Build a relational database for an e-commerce or retail business that stores customers, products, orders, order items, and payments. Use SQL to manage transactional data and generate business reports such as revenue, top products, customer performance, monthly sales, and repeat purchases.

Database: PostgreSQL / MySQL / SQL Server

Core Skills: DDL, DML, joins, GROUP BY, subqueries, CTEs, window functions, views, indexes, constraints, transactions, and query optimization.

Database Structure

Table	Main Columns
customers	customer_id, name, email, city, signup_date
products	product_id, product_name, category, price, stock
orders	order_id, customer_id, order_date, status, total_amount
order_items	order_item_id, order_id, product_id, quantity, unit_price
payments	payment_id, order_id, payment_date, amount, payment_status

How to Approach the Project

1. Understand the Business Requirements

Identify what the business needs to track: customers, products, orders, payments, revenue, and product performance.

2. Design the Database

Create normalized tables and define primary keys and foreign keys between customers, orders, order items, products, and payments.

3. Create Tables

Use CREATE TABLE statements with appropriate data types, NOT NULL rules, UNIQUE constraints, CHECK constraints, primary keys, and foreign keys.

4. Insert Sample Data

Populate realistic customers, products, orders, order items, and payment records using INSERT statements.

5. Perform CRUD Operations

Practice INSERT, SELECT, UPDATE, and DELETE while maintaining data integrity.

6. Write Joins

Use INNER JOIN, LEFT JOIN, and multi-table joins to combine customer, order, product, and payment information.

7. Build Business Analytics

Use SUM, COUNT, AVG, MIN, MAX, GROUP BY, and HAVING to calculate revenue, order volume, average order value, and product performance.

8. Use Advanced SQL

Add subqueries, CTEs, CASE expressions, window functions, ranking, running totals, and month-over-month analysis.

9. Create Views

Create reusable views for dashboards and recurring business reports.

10. Optimize Queries

Analyze execution plans and add indexes to frequently filtered or joined columns when appropriate.

Example SQL: Create Tables

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE,  
  city VARCHAR(80),  
  signup_date DATE  
);  
  
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATE,  
  status VARCHAR(30),  
  total_amount DECIMAL(12,2),  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

Example SQL: Revenue by Month

```
SELECT  
  EXTRACT(YEAR FROM order_date) AS year,  
  EXTRACT(MONTH FROM order_date) AS month,  
  SUM(total_amount) AS revenue  
FROM orders  
WHERE status = 'COMPLETED'  
GROUP BY 1, 2  
ORDER BY 1, 2;
```

Example SQL: Top Products

```
SELECT p.product_name,  
  SUM(oi.quantity) AS units_sold,  
  SUM(oi.quantity * oi.unit_price) AS revenue  
FROM order_items oi  
JOIN products p ON p.product_id = oi.product_id  
GROUP BY p.product_name  
ORDER BY revenue DESC  
LIMIT 10;
```

Example SQL: Customer Ranking

```
SELECT customer_id,  
  SUM(total_amount) AS revenue,  
  RANK() OVER (ORDER BY SUM(total_amount) DESC) AS revenue_rank  
FROM orders  
WHERE status = 'COMPLETED'  
GROUP BY customer_id;
```

Key SQL Concepts Demonstrated

Category	Skills
Database Design	Primary keys, foreign keys, normalization, constraints
Basic SQL	SELECT, INSERT, UPDATE, DELETE
Joins	INNER JOIN, LEFT JOIN, multi-table joins
Aggregation	COUNT, SUM, AVG, GROUP BY, HAVING
Advanced SQL	CTEs, subqueries, CASE, window functions
Analytics	Ranking, running totals, monthly trends, customer segmentation
Performance	Indexes, EXPLAIN/query plans, query optimization
Reusable Reporting	Views and stored procedures where supported

End-to-End Project Flow

Business Requirements → ER/Data Model → Create Tables → Insert Data → CRUD → Joins → Aggregations → Advanced Analytics → Views → Indexing → Query Optimization → Reporting

Why This Is a Strong SQL Project

This project demonstrates both practical database development and analytical SQL. It gives you strong interview examples around joins, aggregations, window functions, data modeling, constraints, performance tuning, and turning raw transactional data into business insights.

Possible Advanced Extensions

- Stored procedures and functions
- Triggers for audit/history tables
- Materialized views
- Partitioning for large order tables
- Index optimization
- Slowly Changing Dimensions for analytics
- ETL pipeline into a data warehouse
- Power BI/Tableau dashboard connected to SQL